INTEGRAL DRAGON

Plan Mode vs. Act Mode

Make the agent show its work before it edits the repo

Session 06 | Module 2: Spec-Driven Development & Context Control

Confidential. Prepared exclusively for Ensemble Health Partners.

CORE PRINCIPLE

Mode separation is an engineering control, not a UI preference

Planning and execution create different risks

The same agent that helps quickly can widen scope quickly. Analysis is safe. Writing to disk is not. Separating the two gives the human a real approval gate — not a notification after the fact.

Read-only analysis is a safety feature

Plan Mode keeps the agent in analysis mode. It can inspect the repo, ask clarifying questions, and propose work without modifying files or executing commands. The first artifact is a plan, not a diff.

The plan is an engineering artifact

A plan should name likely touched files, hidden risks, migration concerns, test implications, and open questions. If it cannot do that, the task is not ready for execution.

When to default to Plan Mode

High-risk or high-ambiguity work

Multi-step implementations, refactors touching many files, migrations, auth, billing, CI, or compliance-sensitive flows. Any work where the repo boundary is still unclear.

Pattern to avoid

"I'll just let it code and fix the bugs later." This is how critical files get overwritten, migrations get corrupted, and schema changes slip through without review.

What Plan Mode is for — and how to start it

Read-only codebase analysis

Explore directory structures, read files, trace dependencies, and understand conventions without any risk of modification.

Clarifying questions & missing-requirement discovery

The agent surfaces what it doesn't know rather than silently inventing answers. This is where ambiguity gets caught.

Multi-file change proposals

Propose which files to touch, what invariants to preserve, and what the diff shape will look like — all before a single byte is written.

Safe exploration of sensitive areas

Auth, billing, migrations, compliance — any area where an accidental write could cause production impact.

```
# Start Claude Code in Plan Mode
$ claude --permission-mode plan

# The agent can now:
✓ Read files and directories
✓ Analyze code and dependencies
✓ Propose changes as a plan
✓ Surface open questions

# The agent cannot:
✗ Edit files
✗ Execute shell commands
✗ Make any changes to disk
```

Review the plan like an engineer, not a spectator

A good plan should contain

Files and components touched

Explicit list of what will be modified. If the plan can't name the files, the task isn't scoped tightly enough.

Invariants that must remain true

Existing behavior that must not break. Schema stability, API contracts, backward compatibility.

Migration or compatibility concerns

Anything that affects deployment, rollback, or data integrity.

Testing strategy and open questions

What test evidence would prove the change is safe? What assumptions remain unresolved?

Questions to challenge the plan

🔍 What did the plan **assume** without evidence? Which business rules were silently invented?

💥 What **failure modes** did it ignore? What happens if the happy path isn't taken?

🚫 Which files **should not** be touched? Are schema files, migration files, or config files in the blast radius?

✅ What **test evidence** would prove this change is safe? Are those tests named in the plan?

Approval is not a rubber stamp

The value is not simply having a plan. The value is **challenging the plan** before execution begins. Reject shallow plans and force refinement.

Plan Mode and permissions solve different problems — in enterprise practice you want both

Claude Code permission modes

Mode	Behavior
`plan`	**Read-only analysis. No writes, no execution.**
`default`	Asks permission before file edits and commands.
`acceptEdits`	Auto-accepts file edits, still asks for commands.
`auto`	Auto-accepts most operations within allowlist.
`bypassPermissions`	No gates. Use only in isolated sandboxes.

Plan Mode controls *when* action begins

It forces the agent to analyze before it acts. The plan is the reviewable artifact that precedes any write.

Permission rules control *what* actions are allowed

Even after planning, permissions scope what the agent can do: file edits, shell commands, network access.

Hooks and managed settings add policy

Enterprise teams layer additional controls: pre-commit hooks, managed settings from IT, CLAUDE.md rules. These compose with permission modes.

Teaching model: The full permission system is richer, but the mental model stays simple — **analyze first, execute only after review.**

EXECUTION RHYTHM

The safest execution rhythm is small and explicit

1. **Spec** — Define what must be true (Session 05)
2. **Plan** — Agent proposes approach in read-only mode
3. **Challenge** — Review, reject, force refinement
4. **Approve** — Explicit go/no-go on the plan
5. **Act** — Execute a bounded slice only
6. **Test** — Verify against acceptance criteria
7. **Narrate** — Document what changed and why

Smaller diffs

Each approved slice is bounded. No more "the AI rewrote half the codebase in one commit."

Earlier reviewer engagement

The reviewer sees the plan before code exists. Feedback happens when it's cheap to change direction.

Clearer test expectations

The plan names invariants and test strategy. Testing is planned, not improvised after the fact.

LIVE DEMO

Propose, critique, refine, approve — live in Claude Code

Demo prompt flow

```
$ claude --permission-mode plan

# Step 1: Propose
> "Analyze the auth module and propose
  the safest refactor path."

# Step 2: Tighten constraints
> "Assume schema changes are not allowed.
  Keep the first PR under 150 lines. Revise."

# Step 3: Require test strategy
> "What tests are required before this
  is safe to merge?"

# Step 4: Approve and execute first slice
> "Execute only the first bounded slice.
  Narrate files changed, risk, and test status."
```

Key teaching points

🔍 The agent can do meaningful work without editing. The first artifact is a plan, not a diff.

🔄 Adding constraints should tighten the plan. If the plan doesn't change when you add constraints, the agent isn't listening.

✅ Approval is not automatic. Challenge-and-revise is part of the workflow, not a sign of failure.

🔪 Do not approve the whole future — approve the next slice. Execution stays small and reviewable.

📝 Narration closes the loop: files changed, risk introduced, tests run or still needed.

FAILURE MODES

Common ways the planning gate breaks down

Approving a plan that never named the files
If the plan says "I'll update the auth module" without listing specific files, it hasn't planned anything. Require file-level specificity before approving.

Executing against an ambiguous request
Skipping planning on vague tickets means the agent fills in the gaps silently. Those silent decisions become surprise diffs in review.

Treating Plan Mode as ceremony
If you always approve the first plan without reading it, you've added a step without adding safety. The value is in the challenge, not the checkbox.

Using broad permissions to skip planning
Jumping to `auto` or `bypassPermissions` because planning "takes too long" trades review quality for speed. In regulated environments, that trade fails.

What success looks like: Smaller diffs, fewer surprise edits, earlier reviewer engagement, clearer test expectations, safer AI use in regulated codebases.

KEY TAKEAWAY

Planning is not delay. Planning is where you move uncertainty out of the repo and into a reviewable artifact.

Do not let the first draft of the agent's thinking become the first write to the repo.

Make the agent show its work. Challenge it. Then approve execution deliberately.

Session 05
Made intent explicit

Session 06
Made execution reviewable

Session 07
Persistent rules via CLAUDE.md